Atividade Acelerômetro – Serial com Threads

Microcontroladores 2 (PSI) – Escola Politécnica da USP
Aluno: Artur Serejo Dantas – Nº USP 15441730
Placa: FRDM-KL25Z · Sensor: MMA8451Q (I²C0, 0x1D) · RTOS: Zephyr

Repositório: https://github.com/arturserejodantas7/acelerometro_threads – branch **atividade-acelerometro-threads**

1. Organização em threads

O projeto foi dividido em três threads com papéis independentes, ligadas por uma fila de mensagens (produtor–consumidor) e por um estado compartilhado protegido por mutex:

Thread	Prioridade	Ativação	Função
t_aquisicao	5 (maior)	periódica 10 ms (k_timer)	Lê os 6 bytes OUT_X..OUT_Z do MMA8451Q, converte para m/s², tira o offset, envia à k_msgq
t_integracao	6	por evento (k_msgq_get)	Integra a → v → x a cada amostra (trapézio; Simpson 1/3 opcional)
t_serial	7 (menor)	periódica 20 ms (k_sleep absoluto)	Copia o estado sob mutex e envia t_ms;ax;ay;az;vx;vy;vz;px;py;pz

O sensor é configurado em ±2 g (4096 contagens/g), ODR de 100 Hz e modo de baixo ruído (CTRL_REG1 = 0x1D). Nos primeiros 2 s a placa deve ficar parada: a média dessas 200 amostras é o offset de cada eixo (inclui a gravidade em z), que é subtraído das leituras seguintes.

2. Estratégia de escalonamento

- **Preemptivo por prioridade fixa.** No Zephyr as três threads são preemptivas (prioridade ≥ 0) e o escalonador sempre executa a thread pronta de maior prioridade (menor número). As prioridades são todas diferentes, então não foi necessário time-slicing (CONFIG_TIMESLICING=n).
- **Atribuição inspirada no Rate Monotonic.** A tarefa de menor período (aquisição, 10 ms) recebe a maior prioridade; a serial (20 ms) a menor. Além do período, pesou a criticidade: a integração supõe Δt constante, então o instante de amostragem precisa ter o menor jitter possível. Com prioridade máxima, quando o k_timer expira a aquisição preempta imediatamente qualquer outra thread.
- **Integração disparada por evento.** A t_integracao fica bloqueada em k_msgq_get e só roda quando chega uma amostra, logo após a aquisição voltar a dormir. Sua prioridade é maior que a da serial para que o cálculo nunca fique atrasado em relação aos dados. A fila de 16 posições absorve atrasos eventuais; a aquisição usa K_NO_WAIT para nunca bloquear e conta amostras perdidas.
- **Serial com a menor prioridade.** O printk de uma linha (~90 caracteres a 115200 bps) ocupa cerca de 8 ms de UART, a tarefa mais longa do sistema. Se tivesse prioridade alta, atrasaria a amostragem; com prioridade baixa ela é simplesmente preemptada e retomada. Por isso também foi decimada para 50 Hz (envia 1 de cada 2 estados), mantendo a ocupação da UART em ~40%. O período usa k_sleep com tempo absoluto (K_TIMEOUT_ABS_MS), para o tempo gasto no printk não se acumular.
- **Sincronização.** O estado (a, v, x) é compartilhado por um k_mutex, cuja herança de prioridade evita inversão de prioridade; a seção crítica é só a cópia de uma struct. Todas as threads bloqueiam (k_timer_status_sync, k_msgq_get, k_sleep) em vez de fazer espera ocupada, de modo que a CPU fica com a thread idle no tempo livre.

- **Utilização estimada.** Leitura I²C (~1 ms a cada 10 ms) ≈ 10%, integração em float por software (o KL25Z não tem FPU) ≈ 2%, serial (~8 ms a cada 20 ms) ≈ 40%: U ≈ 0,52, abaixo do limite de Liu & Layland para 3 tarefas (≈ 0,78), portanto o conjunto é escalonável com folga.

3. Integração numérica

Foi usada a **regra dos trapézios**, aplicada duas vezes: $v[n] = v[n-1] + \Delta t/2 \cdot (a[n-1] + a[n])$ e $x[n] = x[n-1] + \Delta t/2 \cdot (v[n-1] + v[n])$, com $\Delta t = 10$ ms. Com INTEGRADOR_SIMPSON = 1 a velocidade passa a ser calculada pela **regra de Simpson 1/3** a cada par de intervalos: $v[n] = v[n-2] + \Delta t/3 \cdot (a[n-2] + 4a[n-1] + a[n])$. A quadratura gaussiana não foi usada porque exige avaliar a função em pontos internos não uniformes, e o sensor fornece amostras em instantes fixos e igualmente espaçados.

A dupla integração amplifica qualquer offset residual (erro de posição cresce com t^2). Para limitar a deriva: (i) calibração do offset na inicialização; (ii) zona morta de 0,05 m/s²; (iii) *zero-velocity update*: se os três eixos ficam abaixo de 0,10 m/s² por 500 ms, a velocidade é zerada.

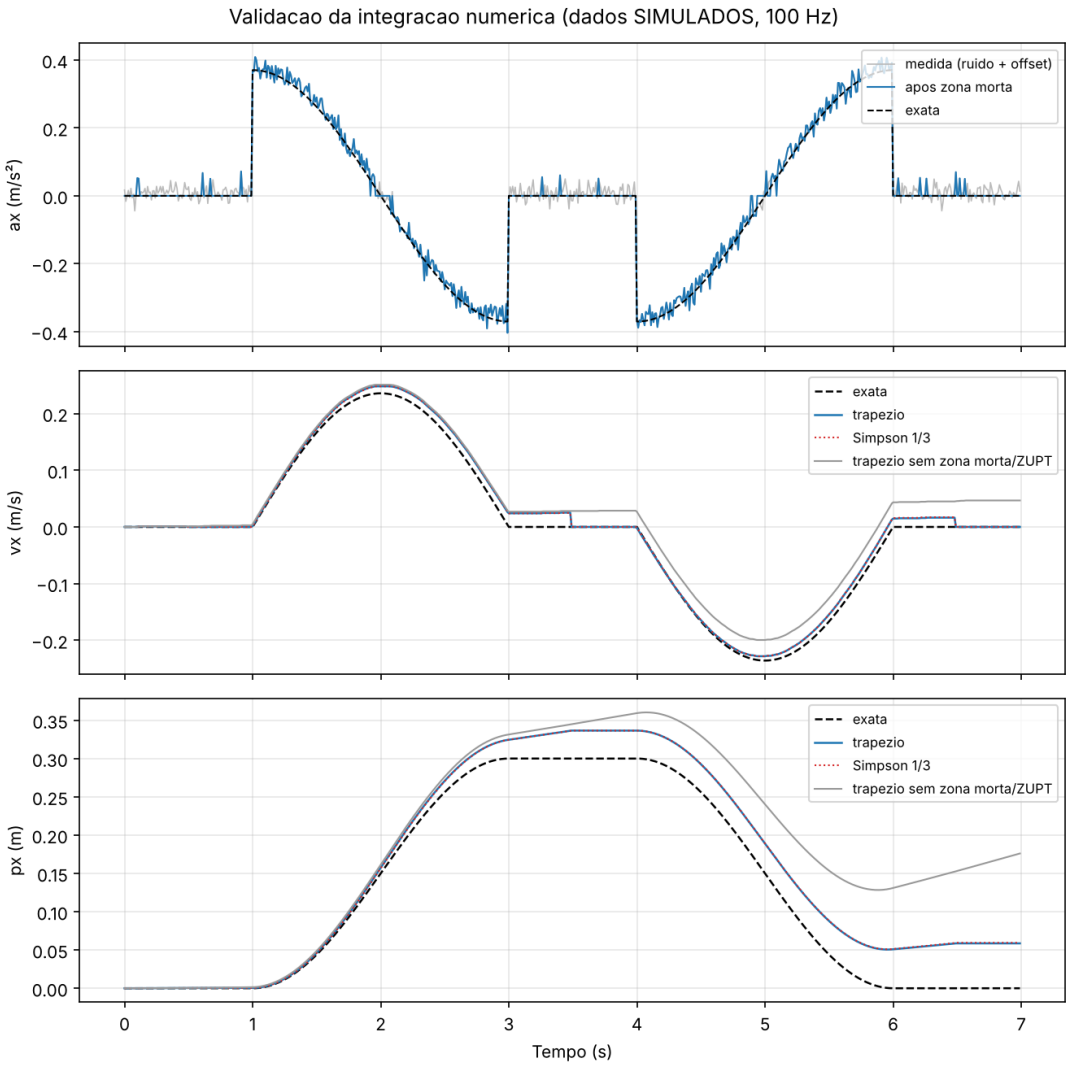


Figura 1 – Validação do algoritmo (mesmas equações do firmware) com movimento conhecido de 0,30 m em x, ruído de 0,02 m/s² e offset residual. Dados simulados.

Na simulação, trapézio e Simpson ficam praticamente sobrepostos: a 100 Hz o erro de truncamento das duas regras é muito menor que o erro causado por ruído e offset. Sem zona

morta/ZUPT, a posição deriva cerca de 18 cm em 7 s; com as correções, o erro final cai para cerca de 6 cm e a posição para de derivar quando a placa para.

4. Aceleração, velocidade e posição

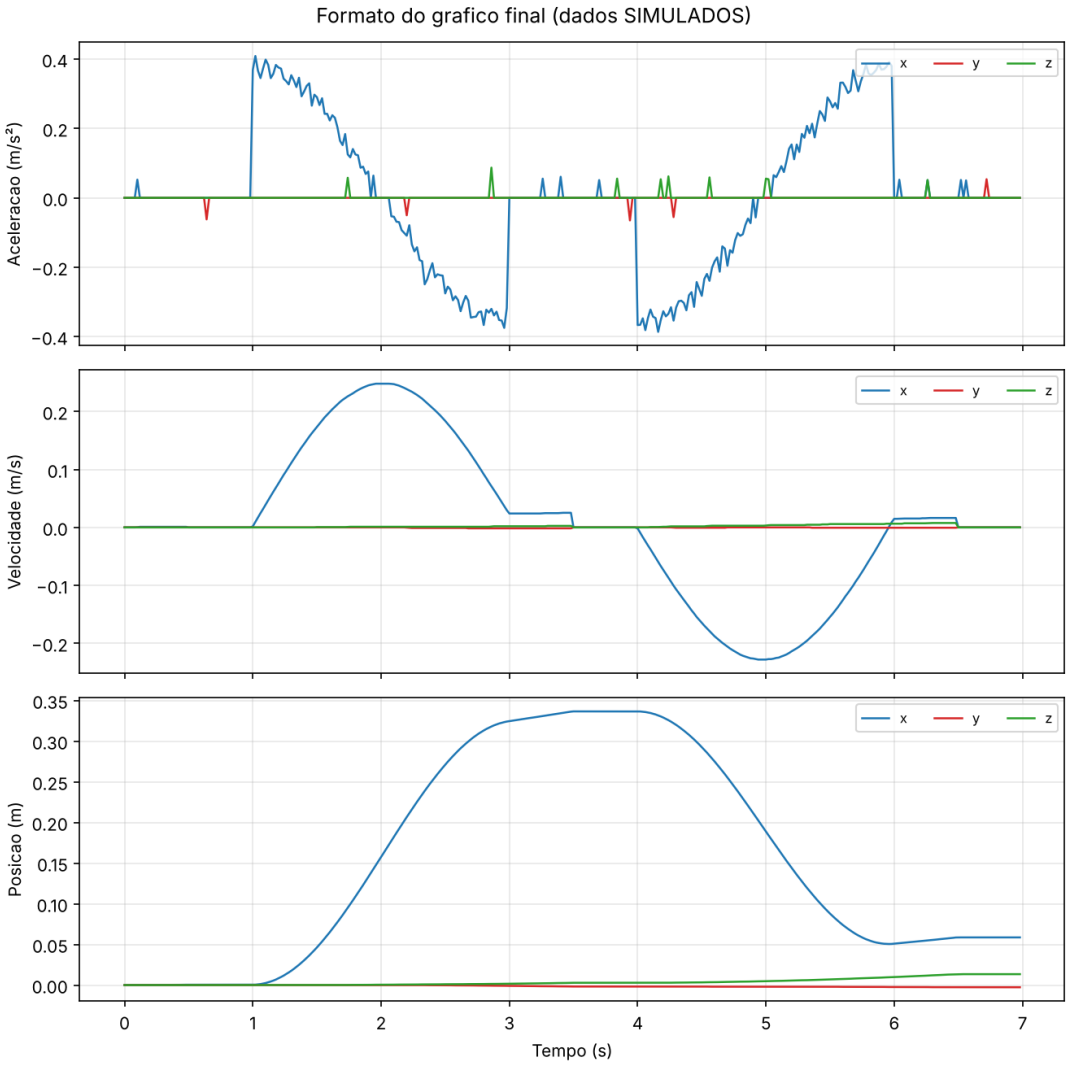


Figura 2 – Formato do gráfico gerado por graficos.py. ATENÇÃO: dados simulados; substituir pelo gráfico dos dados reais da placa.

Fluxo de captura: `python python/leitor_serial.py COM3 dados.csv` grava t, a, v e x em CSV; `python python/graficos.py dados.csv grafico.png` gera a figura.

5. Código-fonte

src/main.c

```
/*
 * Atividade Acelerometro - Serial (com threads)
 * Placa: FRDM-KL25Z  |  Sensor: MMA8451Q no I2C0 (endereco 0x1D)
 * RTOS: Zephyr
 *
 * Arquitetura (produtor -> consumidor):
 *
 *   [t_aquisicao]  prio 5, periodica 10 ms (100 Hz)
 *        |  le o MMA8451Q, converte para m/s^2, remove o offset
 *        v  k_msgq q_amostras
 *   [t_integracao] prio 6, liberada por evento (chegada de amostra)
 *        |  integra a -> v -> x (trapezio ou Simpson 1/3)
 *        v  estado compartilhado protegido por k_mutex
 *   [t_serial]     prio 7, periodica 20 ms (50 Hz)
 *           envia "t_ms;ax;ay;az;vx;vy;vz;px;py;pz" pela UART0 (OpenSDA)
 *
 * No Zephyr, numero de prioridade MENOR = prioridade MAIOR.
 * Todas sao threads preemptivas (prioridade >= 0).
 */

#include <zephyr/kernel.h>
#include <zephyr/device.h>
#include <zephyr/drivers/i2c.h>
#include <zephyr/sys/printk.h>

#define ABSF(x) ((x) < 0.0f ? -(x) : (x))
#include <string.h>

/* ------------------------------------------------------------------ */
/* Configuracao                                                       */
/* ------------------------------------------------------------------ */
#define MMA_ADDR            0x1D
#define MMA_REG_STATUS      0x00
#define MMA_REG_OUT_X_MSB   0x01
#define MMA_REG_WHO_AM_I    0x0D
#define MMA_REG_XYZ_CFG     0x0E
#define MMA_REG_CTRL1       0x2A
#define MMA_WHO_AM_I_VAL    0x1A

#define G_MS2               9.80665f
#define CONTAGENS_POR_G     4096.0f     /* faixa +-2 g, 14 bits */

#define PERIODO_AQ_MS       10          /* 100 Hz */
#define PERIODO_TX_MS       20          /* 50 Hz  */
#define DT_S                (PERIODO_AQ_MS / 1000.0f)

#define N_CALIBRACAO        200         /* 2 s parado para medir o offset */

/* Metodo de integracao: 0 = trapezio, 1 = Simpson 1/3 */
#define INTEGRADOR_SIMPSON  0

/* Reduz a deriva da dupla integracao */
#define ZONA_MORTA_MS2      0.05f       /* |a| menor que isso vira 0          */
#define LIMIAR_PARADO_MS2   0.10f       /* |a| < isso nos 3 eixos = "parado"  */
#define N_PARADO_ZERA_V     50          /* 500 ms parado -> zera velocidade   */

#define PRIO_AQ             5
#define PRIO_INT            6
#define PRIO_TX             7
#define PILHA_AQ            1024
#define PILHA_INT           1024
#define PILHA_TX            1536        /* printk com float usa mais pilha */

/* ------------------------------------------------------------------ */
/* Tipos e objetos do kernel                                          */
/* ------------------------------------------------------------------ */
struct amostra {
    uint32_t t_ms;
    float a[3];          /* m/s^2, ja sem offset */
};

struct estado {
    uint32_t t_ms;
    float a[3];
    float v[3];          /* m/s */
    float p[3];          /* m   */
};

K_MSGQ_DEFINE(q_amostras, sizeof(struct amostra), 16, 4);
K_MUTEX_DEFINE(mtx_estado);
```

```
K_SEM_DEFINE(sem_pronto, 0, 1);       /* libera integracao e serial apos calibrar */

static struct estado estado_global;
static const struct device *i2c_dev = DEVICE_DT_GET(DT_NODELABEL(i2c0));
static float offset[3];
static volatile uint32_t amostras_perdidas;

/* ------------------------------------------------------------------ */
/* Driver simples do MMA8451Q                                          */
/* ------------------------------------------------------------------ */
static int mma_init(void)
{
    uint8_t id;
    int ret;

    if (!device_is_ready(i2c_dev)) {
        printk("I2C0 nao esta pronto\n");
        return -ENODEV;
    }
    ret = i2c_reg_read_byte(i2c_dev, MMA_ADDR, MMA_REG_WHO_AM_I, &id);
    if (ret || id != MMA_WHO_AM_I_VAL) {
        printk("MMA8451Q nao encontrado (ret=%d id=0x%02x)\n", ret, id);
        return -EIO;
    }
    /* Standby para configurar */
    i2c_reg_write_byte(i2c_dev, MMA_ADDR, MMA_REG_CTRL1, 0x00);
    /* Faixa +-2 g */
    i2c_reg_write_byte(i2c_dev, MMA_ADDR, MMA_REG_XYZ_CFG, 0x00);
    /* ODR = 100 Hz (DR=011), LNOISE=1, ACTIVE=1 -> 0b0001_1101 */
    i2c_reg_write_byte(i2c_dev, MMA_ADDR, MMA_REG_CTRL1, 0x1D);
    return 0;
}

static int mma_ler(float a_ms2[3])
{
    uint8_t b[6];
    int ret = i2c_burst_read(i2c_dev, MMA_ADDR, MMA_REG_OUT_X_MSB, b, 6);

    if (ret) {
        return ret;
    }
    for (int i = 0; i < 3; i++) {
        int16_t bruto = (int16_t)((b[2 * i] << 8) | b[2 * i + 1]) >> 2; /* 14 bits */
        a_ms2[i] = (bruto / CONTAGENS_POR_G) * G_MS2;
    }
    return 0;
}

/* ------------------------------------------------------------------ */
/* Thread 1: aquisicao (periodica, maior prioridade)                   */
/* ------------------------------------------------------------------ */
static void t_aquisicao(void *p1, void *p2, void *p3)
{
    struct k_timer tmr;
    struct amostra s;
    float a[3], soma[3] = {0};

    if (mma_init() != 0) {
        return;
    }

    k_timer_init(&tmr, NULL, NULL);
    k_timer_start(&tmr, K_MSEC(PERIODO_AQ_MS), K_MSEC(PERIODO_AQ_MS));

    /* Calibracao: placa parada sobre a mesa.
     * O offset inclui a gravidade em z (aprox. +1 g). */
    printk("# Calibrando, mantenha a placa parada...\n");
    for (int n = 0; n < N_CALIBRACAO; n++) {
        k_timer_status_sync(&tmr);
        if (mma_ler(a) == 0) {
            for (int i = 0; i < 3; i++) {
                soma[i] += a[i];
            }
        }
    }
    for (int i = 0; i < 3; i++) {
        offset[i] = soma[i] / N_CALIBRACAO;
    }
    printk("# Offset: %.3f %.3f %.3f m/s2\n",
           (double)offset[0], (double)offset[1], (double)offset[2]);
    k_sem_give(&sem_pronto);

    while (1) {
        /* Bloqueia ate o proximo periodo: libera a CPU para as outras threads */
```

```
        k_timer_status_sync(&tmr);

        if (mma_ler(a) != 0) {
            continue;
        }
        s.t_ms = k_uptime_get_32();
        for (int i = 0; i < 3; i++) {
            s.a[i] = a[i] - offset[i];
        }
        /* K_NO_WAIT: a thread de tempo real nunca bloqueia esperando o consumidor */
        if (k_msgq_put(&q_amostras, &s, K_NO_WAIT) != 0) {
            amostras_perdidas++;
        }
    }
}

/* ------------------------------------------------------------------ */
/* Thread 2: integracao numerica (liberada por evento)                 */
/* ------------------------------------------------------------------ */
static void t_integracao(void *p1, void *p2, void *p3)
{
    struct amostra s;
    float a0[3] = {0}, a1[3] = {0};   /* a[n-2], a[n-1] */
    float v0[3] = {0}, v1[3] = {0};   /* v[n-2], v[n-1] */
    float v[3] = {0}, p[3] = {0};
    int parado = 0;
    uint32_t n = 0;

    k_sem_take(&sem_pronto, K_FOREVER);
    k_sem_give(&sem_pronto);              /* repassa para a thread serial */

    while (1) {
        k_msgq_get(&q_amostras, &s, K_FOREVER);

        /* Zona morta: elimina ruido residual em torno de zero */
        bool quieto = true;
        for (int i = 0; i < 3; i++) {
            if (ABSF(s.a[i]) >= LIMIAR_PARADO_MS2) {
                quieto = false;
            }
            if (ABSF(s.a[i]) < ZONA_MORTA_MS2) {
                s.a[i] = 0.0f;
            }
        }

        if (n == 0) {
            /* primeira amostra: so guarda */
        } else {
            for (int i = 0; i < 3; i++) {
                float vn;
#if INTEGRADOR_SIMPSON
                if ((n % 2) == 0) {
                    /* Simpson 1/3 sobre os 2 ultimos intervalos:
                     * v[n] = v[n-2] + dt/3 (a[n-2] + 4 a[n-1] + a[n]) */
                    vn = v0[i] + (DT_S / 3.0f) * (a0[i] + 4.0f * a1[i] + s.a[i]);
                } else {
                    /* amostra impar: trapezio no intervalo isolado */
                    vn = v1[i] + 0.5f * DT_S * (a1[i] + s.a[i]);
                }
#else
                /* Regra dos trapezios: v[n] = v[n-1] + dt/2 (a[n-1] + a[n]) */
                vn = v1[i] + 0.5f * DT_S * (a1[i] + s.a[i]);
#endif
                /* posicao: trapezio sobre a velocidade */
                p[i] += 0.5f * DT_S * (v1[i] + vn);
                v[i] = vn;
            }
        }

        /* "Zero-velocity update": parado por um tempo -> v = 0 */
        parado = quieto ? parado + 1 : 0;
        if (parado >= N_PARADO_ZERA_V) {
            for (int i = 0; i < 3; i++) {
                v[i] = 0.0f;
            }
        }

        for (int i = 0; i < 3; i++) {
            a0[i] = a1[i];
            a1[i] = s.a[i];
            v0[i] = v1[i];
            v1[i] = v[i];
        }
        n++;
```

```
        k_mutex_lock(&mtx_estado, K_FOREVER);
        estado_global.t_ms = s.t_ms;
        memcpy(estado_global.a, s.a, sizeof(s.a));
        memcpy(estado_global.v, v, sizeof(v));
        memcpy(estado_global.p, p, sizeof(p));
        k_mutex_unlock(&mtx_estado);
    }
}

/* ------------------------------------------------------------------ */
/* Thread 3: envio pela serial (periodica, menor prioridade)           */
/* ------------------------------------------------------------------ */
static void t_serial(void *p1, void *p2, void *p3)
{
    struct estado e;
    uint32_t ultimo_t = 0;
    int64_t prox;

    k_sem_take(&sem_pronto, K_FOREVER);
    printk("t_ms;ax;ay;az;vx;vy;vz;px;py;pz\n");

    prox = k_uptime_get();
    while (1) {
        prox += PERIODO_TX_MS;
        /* Espera absoluta: o periodo nao acumula o tempo gasto no printk */
        k_sleep(K_TIMEOUT_ABS_MS(prox));

        k_mutex_lock(&mtx_estado, K_FOREVER);
        e = estado_global;              /* copia rapida, solta o mutex logo */
        k_mutex_unlock(&mtx_estado);

        if (e.t_ms == ultimo_t) {
            continue;                   /* nada novo */
        }
        ultimo_t = e.t_ms;

        printk("%u;%.3f;%.3f;%.3f;%.4f;%.4f;%.4f;%.4f;%.4f;%.4f\n",
               e.t_ms,
               (double)e.a[0], (double)e.a[1], (double)e.a[2],
               (double)e.v[0], (double)e.v[1], (double)e.v[2],
               (double)e.p[0], (double)e.p[1], (double)e.p[2]);
    }
}

K_THREAD_DEFINE(id_aq,  PILHA_AQ,  t_aquisicao,  NULL, NULL, NULL, PRIO_AQ,  0, 0);
K_THREAD_DEFINE(id_int, PILHA_INT, t_integracao, NULL, NULL, NULL, PRIO_INT, 0, 0);
K_THREAD_DEFINE(id_tx,  PILHA_TX,  t_serial,     NULL, NULL, NULL, PRIO_TX,  0, 0);

int main(void)
{
    /* Toda a aplicacao roda nas threads acima; main pode terminar. */
    return 0;
}
```

prj.conf

```
# Atividade Acelerometro - Serial com threads (FRDM-KL25Z)
CONFIG_I2C=y
CONFIG_PRINTK=y
CONFIG_SERIAL=y
CONFIG_CONSOLE=y
CONFIG_UART_CONSOLE=y

# printk com %f (o KL25Z nao tem FPU: float por software)
CONFIG_CBPRINTF_FP_SUPPORT=y

# Escalonamento: preemptivo por prioridade fixa (padrao do Zephyr)
CONFIG_NUM_PREEMPT_PRIORITIES=15
CONFIG_TIMESLICING=n
CONFIG_SYS_CLOCK_TICKS_PER_SEC=1000
CONFIG_TIMEOUT_64BIT=y

CONFIG_THREAD_NAME=y
CONFIG_MAIN_STACK_SIZE=1024
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20.0)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(acelerometro_threads)

target_sources(app PRIVATE src/main.c)
```

python/leitor_serial.py

```
"""
```

```
Leitor serial - Atividade Acelerometro (FRDM-KL25Z)

Captura as linhas enviadas pela placa no formato
    t_ms;ax;ay;az;vx;vy;vz;px;py;pz
e grava em um arquivo .csv com aceleracao, velocidade e posicao.

Uso:
    pip install pyserial
    python leitor_serial.py                # usa PORTA e ARQUIVO abaixo
    python leitor_serial.py COM5 dados.csv  # ou passe pela linha de comando

Pare com Ctrl+C.
"""
import csv
import sys
import time

import serial

PORTA = "COM3"            # Windows: COMx (veja no Gerenciador de Dispositivos)
                          # Linux: /dev/ttyACM0  |  macOS: /dev/cu.usbmodemXXXX
BAUD = 115200
ARQUIVO = "dados_acelerometro.csv"

CABECALHO = ["t_s", "ax", "ay", "az", "vx", "vy", "vz", "px", "py", "pz"]


def main():
    porta = sys.argv[1] if len(sys.argv) > 1 else PORTA
    arquivo = sys.argv[2] if len(sys.argv) > 2 else ARQUIVO

    ser = serial.Serial(porta, BAUD, timeout=1)
    print(f"Lendo {porta} @ {BAUD} -> {arquivo}  (Ctrl+C para parar)")

    t0 = None
    n = 0
    with open(arquivo, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(CABECALHO)
        try:
            while True:
                linha = ser.readline().decode("ascii", errors="ignore").strip()
                if not linha:
                    continue
                if linha.startswith("#") or linha.startswith("t_ms"):
                    print(linha)          # mensagens de calibracao / cabecalho
                    continue
                campos = linha.split(";")
                if len(campos) != 10:
                    continue              # linha incompleta
                try:
                    valores = [float(c) for c in campos]
                except ValueError:
                    continue
                if t0 is None:
                    t0 = valores[0]
                valores[0] = (valores[0] - t0) / 1000.0   # ms -> s, comeca em 0
                w.writerow([f"{v:.4f}" for v in valores])
                n += 1
                if n % 50 == 0:
                    f.flush()
                    print(f"{n} amostras | a=({valores[1]:+.2f},{valores[2]:+.2f},{valores[3]:+.2f}) "
                          f"v=({valores[4]:+.3f},{valores[5]:+.3f},{valores[6]:+.3f}) "
                          f"p=({valores[7]:+.3f},{valores[8]:+.3f},{valores[9]:+.3f})")
        except KeyboardInterrupt:
            pass
    ser.close()
    print(f"\n{n} amostras salvas em {arquivo}")


if __name__ == "__main__":
    main()
```

python/graficos.py

```
"""
Gera o grafico de aceleracao, velocidade e posicao a partir do .csv
gravado pelo leitor_serial.py.

Uso:
    pip install matplotlib
    python graficos.py dados_acelerometro.csv          # salva grafico.png
    python graficos.py dados_acelerometro.csv saida.png
"""
import csv
```

```
import sys

import matplotlib.pyplot as plt


def carregar(arquivo):
    cols = {k: [] for k in ["t_s", "ax", "ay", "az", "vx", "vy", "vz", "px", "py", "pz"]}
    with open(arquivo, newline="") as f:
        for linha in csv.DictReader(f):
            for k in cols:
                cols[k].append(float(linha[k]))
    return cols


def plotar(d, saida, titulo="Acelerometro MMA8451Q - FRDM-KL25Z"):
    fig, ax = plt.subplots(3, 1, figsize=(9, 9), sharex=True)
    cores = {"x": "#1f77b4", "y": "#d62728", "z": "#2ca02c"}
    grandezas = [("a", "Aceleracao (m/s²)"), ("v", "Velocidade (m/s)"), ("p", "Posicao (m)")]
    for eixo, (g, rot) in zip(ax, grandezas):
        for c in "xyz":
            eixo.plot(d["t_s"], d[g + c], label=c, color=cores[c], lw=1.2)
        eixo.set_ylabel(rot)
        eixo.grid(alpha=0.3)
        eixo.legend(loc="upper right", ncol=3, fontsize=8)
    ax[-1].set_xlabel("Tempo (s)")
    fig.suptitle(titulo)
    fig.tight_layout()
    fig.savefig(saida, dpi=150)
    print(f"Grafico salvo em {saida}")


if __name__ == "__main__":
    arq = sys.argv[1] if len(sys.argv) > 1 else "dados_acelerometro.csv"
    out = sys.argv[2] if len(sys.argv) > 2 else "grafico.png"
    plotar(carregar(arq), out)
    plt.show()
```